

Experiment No.	9
Experiment Title	Perceptron vs Multilayer Perceptron (A/B Experiment) with Hyperparameter Tuning ¹
Student Name	Sanjay S
Register Number	3122247001056
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	20/09/2026

1 Objective

Build a Perceptron (PLA) and a Multilayer Perceptron (MLP) from scratch, tune the MLP properly, and compare the two on handwritten character recognition.

2 Problem Statement

Classify 3,410 handwritten character images into 62 classes (0–9, A–Z, a–z).

- **Model A:** single layer Perceptron, step activation.
- **Model B:** MLP with hidden layers, trained with backpropagation.
- Compare them on a held out test set.

3 Dataset Description

Dataset	Source	Samples	Features	Classes	Missing
English Hand-written Characters	Kaggle, dhruvildave/english-handwritten-characters-dataset	3,410	784 (28x28 pixels)	62	none

- 55 images per class, so the data is balanced.
- Split 70/15/15, stratified, seed 42: 2387 train, 511 validation, 512 test.
- Validation set used for all tuning, test set used once at the end.

¹GitHub Repository: <https://github.com/sanjay-6727/Introduction-to-Machine-Learning->

4 Exploratory Data Analysis

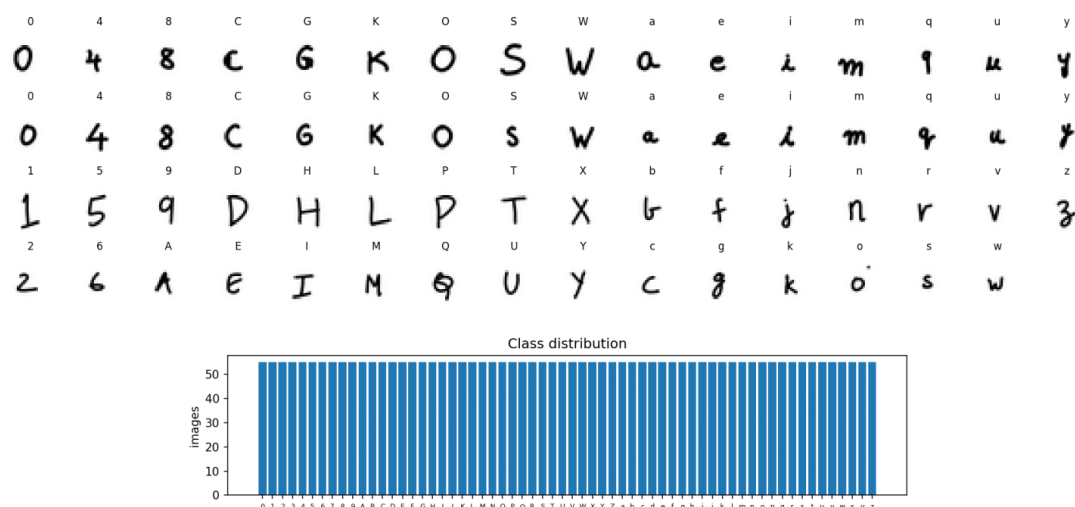


Figure 1: Sample images (top) and class distribution (bottom).

Inference:

- Every class has exactly 55 images, so chance level is only 1/62 (about 1.6%).
- Handwriting varies a lot inside one class (thick or thin, slanted or upright).
- Some characters look almost the same at 28x28: 0/O/o and 1/I/l. This makes the problem hard.

5 Data Preprocessing

- Convert to grayscale, pad to a square, resize to 28x28.
- Invert so ink = 1, then scale each image to [0, 1].
- Flatten to 784 values.
- Standardize each pixel using the training set mean and standard deviation.

Listing 1: Image preprocessing

```

1 def prep_image(pil_img):
2     """grayscale -> 28x28 -> ink=1 / background=0 -> per-image
   min-max scaling."""
3     g = np.asarray(pil_img.convert("L").resize((IMG, IMG), Image.
   LANCZOS), dtype=np.float32) / 255.0
4     g = 1.0 - g
5     g = (g - g.min()) / (g.max() - g.min() + 1e-8)
6     return g

```

6 Machine Learning Algorithm

Both models are written in NumPy. scikit-learn is only used for the split and the metrics.

Model A: Perceptron (PLA)

- Step activation: $\hat{y} = \text{step}(w^T x + b)$
- Update rule: $w_{t+1} = w_t + \eta (y - \hat{y})x$
- 62 one vs rest perceptrons, prediction = class with the highest score.
- Fast and simple, but only draws straight boundaries.

Model B: MLP

- Input, hidden layers with ReLU / tanh / sigmoid, softmax output.
- Loss: cross entropy (MSE also tested). Trained with backpropagation.
- Optimizers: Gradient Descent, SGD, Adam.
- Learns non linear boundaries, but has many hyperparameters and overfits easily.

Listing 2: PLA from scratch

```

1 class PLA:
2     """62 independent perceptrons (one per class). Step
3     activation during training,
4     update w <- w + eta (y - y_hat) x applied sample-by-sample
5     (online). Prediction for the
6     multi-class decision = argmax of the raw scores w_k.x."""
7
8     def __init__(self, n_in, n_cls, eta=0.1, seed=SEED):
9         self.W = np.zeros((n_in + 1, n_cls), np.float32)
10        self.eta, self.rng = eta, np.random.default_rng(seed)
11
12    @staticmethod
13    def _aug(X): return np.hstack([X, np.ones((len(X), 1), np.
14        float32)])
15
16    def scores(self, X): return self._aug(X) @ self.W
17
18    def predict(self, X): return self.scores(X).argmax(1)
19
20    def fit(self, X, y, epochs=50, Xv=None, yv=None):
21        Xa = self._aug(X)
22        T = np.eye(self.W.shape[1], dtype=np.float32)[y]
23        hist = {"train_err": [], "val_err": []}
24        for ep in range(epochs):
25            for i in self.rng.permutation(len(Xa)):
26                yhat = (Xa[i] @ self.W >= 0).astype(np.float32)
27                # step activation
28                self.W += self.eta * np.outer(Xa[i], T[i] - yhat)
29                # PLA update rule
30            hist["train_err"].append(float((self.predict(X) != y)
31                .mean()))
32            if Xv is not None:

```

```

27         hist["val_err"].append(float((self.predict(Xv) !=
28         return hist

```

Listing 3: One MLP training step (backpropagation and optimizer update)

```

1  def _step(self, Xb, Yb):
2      A, Z = self.forward(Xb)
3      n = len(Xb)
4      P = A[-1]
5      if self.loss_name == "ce":
6          d = (P - Yb) / n                                     #
7          softmax + cross-entropy
8      else:
9          g = 2 * (P - Yb) / n                                   #
10         MSE through softmax Jacobian
11         d = P * (g - np.sum(g * P, 1, keepdims=True))
12         gW, gb = [None] * len(self.W), [None] * len(self.W)
13         for i in reversed(range(len(self.W))):
14             gW[i] = A[i].T @ d + self.l2 * self.W[i]
15             gb[i] = d.sum(0)
16             if i > 0:
17                 d = (d @ self.W[i].T) * act_d(self.act, A[i], Z[i]
18                 - 1])
19         params, grads = self.W + self.b, gW + gb
20         if self.opt in ("gd", "sgd"):
21             for p, g in zip(params, grads): p -= self.lr * g
22         elif self.opt == "adam":
23             self.t += 1
24             b1, b2, eps = 0.9, 0.999, 1e-8
25             for k, (p, g) in enumerate(zip(params, grads)):
26                 self.m[k] = b1 * self.m[k] + (1 - b1) * g
27                 self.v[k] = b2 * self.v[k] + (1 - b2) * g * g
28                 mh = self.m[k] / (1 - b1 ** self.t)
29                 vh = self.v[k] / (1 - b2 ** self.t)
30                 p -= self.lr * mh / (np.sqrt(vh) + eps)

```

7 Experimental Setup

Python Version	3.12.3
NumPy Version	2.4.4
Scikit-Learn Version	1.8.0
Operating System	Ubuntu Linux, CPU only
Random Seed	42
Hardware	

8 Hyperparameter Selection

PLA: the learning rate η makes no difference. With weights starting at zero, η only scales the weights, and the step function and argmax ignore scale. Used $\eta = 0.01$.

MLP: tuned one factor at a time on validation accuracy, keeping the best value each time. Starting point: 1 hidden layer of 128 ReLU units, cross entropy, Adam, lr 10^{-3} , batch 64, 60 epochs.

Factor	Settings tried	Best setting	Best val. acc.	Worst val. acc.
Optimizer + learning rate	11	sgd lr=0.5	45.6%	15.9%
Activation	3	relu	45.6%	41.1%
Cost function	2	cross-entropy	45.6%	43.6%
Batch size	5	bs=32	47.7%	25.6%
Depth / width + lr	10	1x256	50.1%	1.6%
L2 weight decay	4	l2=0.001	53.6%	48.1%

Table 2: Tuning summary. Detailed tables are in Section 11.

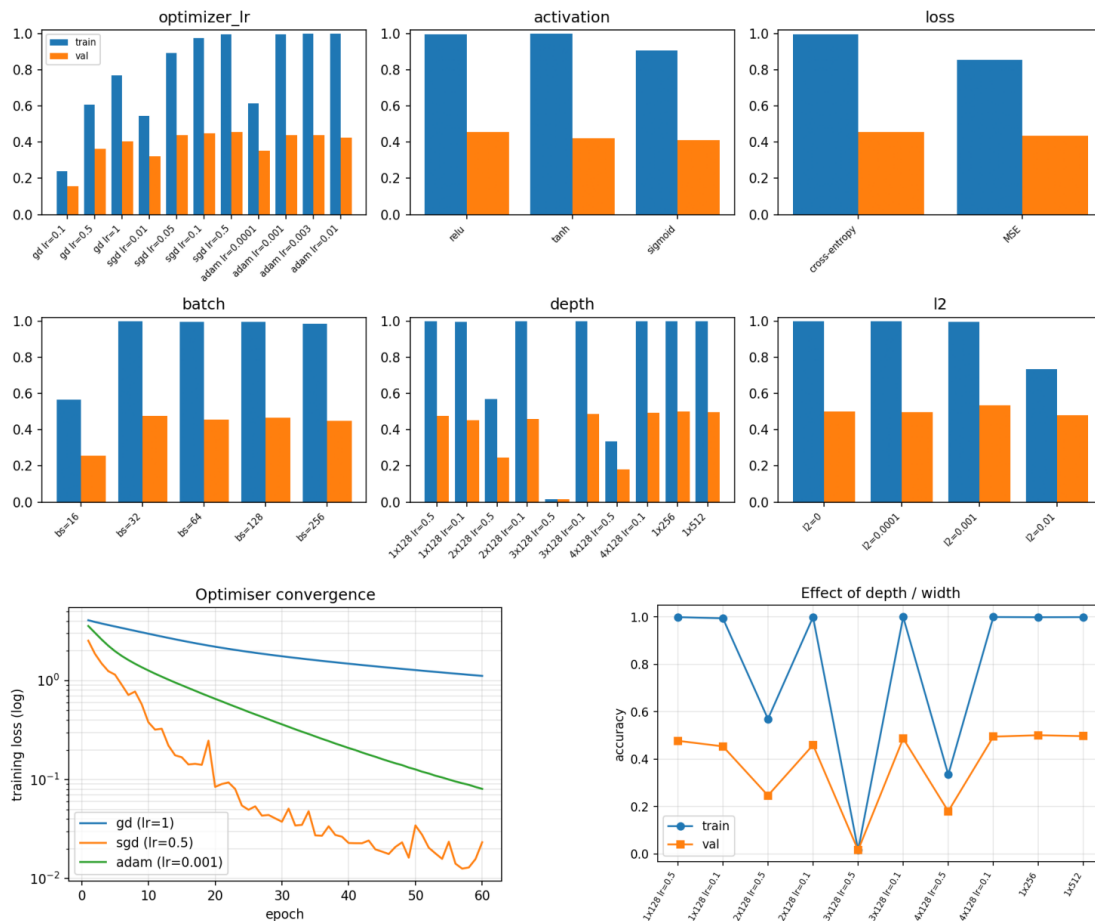


Figure 2: Train vs validation accuracy for every tuning stage (top), optimizer convergence and depth (bottom).

Hyperparameter	Chosen	Why
Hidden layers	1 x 256	Best validation accuracy (50.1%). More layers did not clearly help.
Activation	ReLU	Best validation accuracy (45.6%).
Cost function	Cross entropy	Better than MSE (45.6% vs 43.6%).
Optimizer, lr	SGD, 0.5	Best of 11 combinations (45.6%). Best Adam: 44.0%.
Batch size	32	Best of 16 to 256 (47.7%).
L2 weight decay	10^{-3}	Best validation accuracy (53.6% vs 50.1% without).
Early stopping	patience 20	Stopped after 119 epochs.

Table 3: Final MLP configuration.

9 Results

Model	Accuracy	Precision	Recall	F1	Macro AUC	Train acc.
A: PLA	23.0%	25.7%	23.0%	23.1%	0.782	76.3%
B: MLP (tuned)	49.4%	50.9%	49.5%	48.8%	0.939	99.7%
MLP (untuned) [†]	1.2%	0.2%	1.2%	0.3%	—	—

Table 4: Test set results (precision, recall and F1 are macro averages).

[†] sigmoid, MSE, SGD, lr 0.01. It stays at chance level.

10 Result Visualization

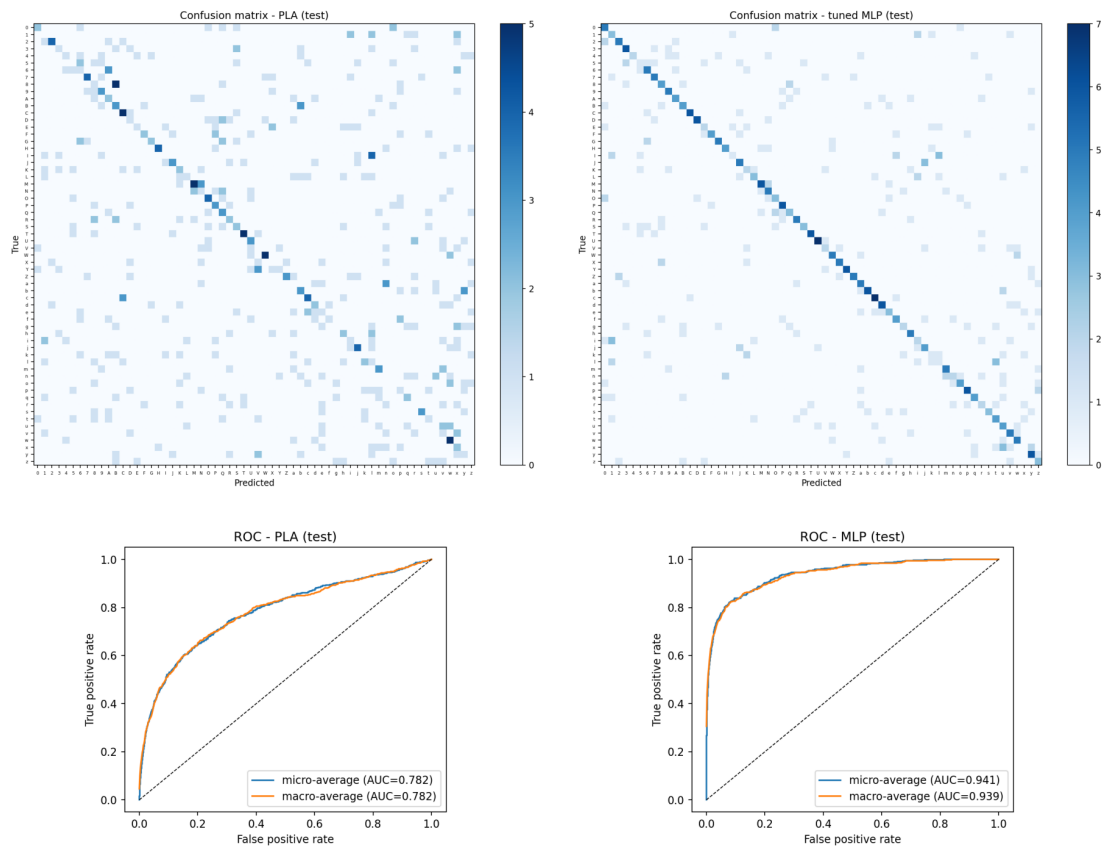


Figure 3: Confusion matrices (PLA left, MLP right) and ROC curves on the test set.

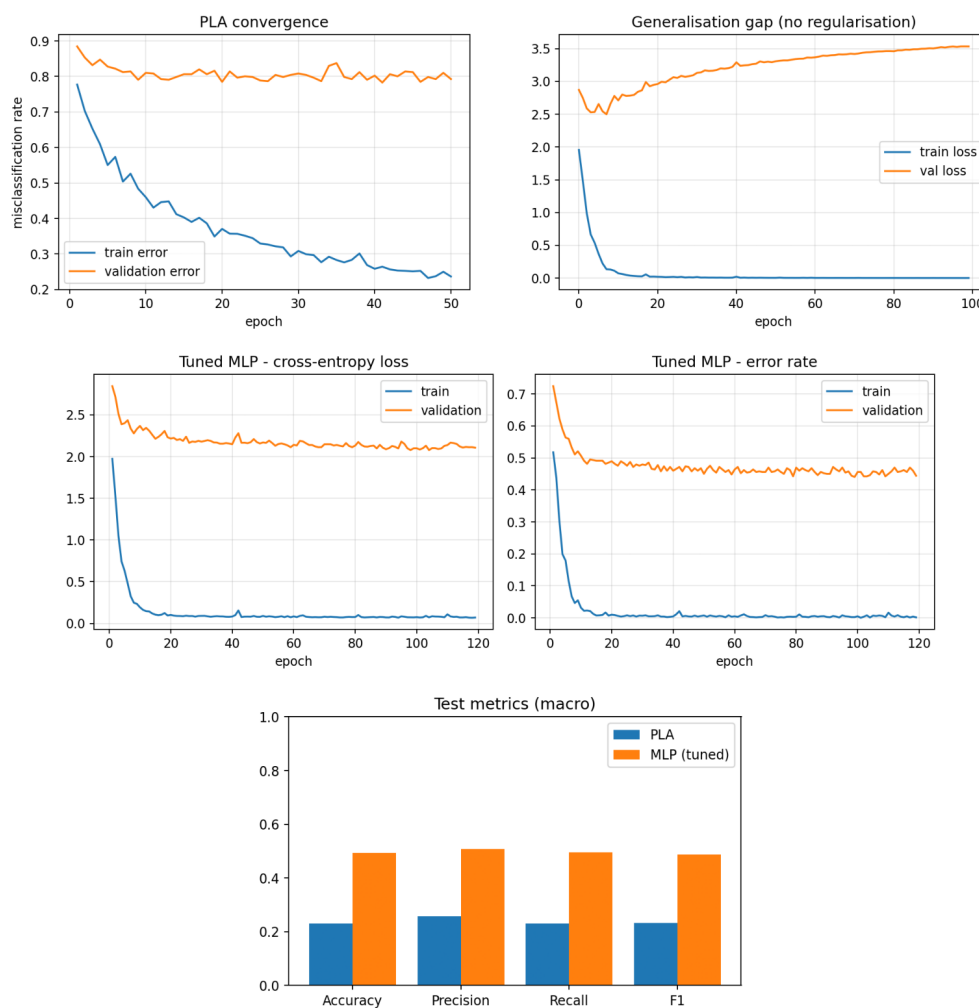


Figure 4: Error and loss against epochs, and test metrics for both models.

Inference:

- PLA confusion matrix is scattered. MLP has a clear diagonal.
- MLP mistakes are mostly look alike: $x \rightarrow u$ (3), $l \rightarrow t$ (3), $i \rightarrow l$ (3), $I \rightarrow l$ (3) (true $\rightarrow$ predicted, count).
- Hardest classes: ‘x’ (0%), ‘I’ (11%), ‘l’ (12%), ‘5’ (12%). Easiest: ‘U’ (88%), ‘c’ (78%), ‘C’ (75%), ‘3’ (75%).
- ROC: MLP is close to the top left corner (AUC 0.939), PLA is much lower (0.782).
- PLA training error stops at about 24% and never reaches zero. MLP training loss goes almost to zero.

11 Detailed Tuning Results

Each run used 60 epochs. “–” means the run never reached exactly zero training error.

Setting	Train acc.	Val. acc.	Epochs to 0 train err.	Final train loss
gd lr=0.1	24.0%	15.9%	–	3.4027
gd lr=0.5	60.6%	36.2%	–	1.7591
gd lr=1	76.8%	40.3%	–	1.1169
sgd lr=0.01	54.7%	32.3%	–	2.0433
sgd lr=0.05	89.2%	43.8%	–	0.5896
sgd lr=0.1	97.7%	44.8%	–	0.2072
sgd lr=0.5	99.5%	45.6%	–	0.0233
adam lr=0.0001	61.3%	35.2%	–	1.7563
adam lr=0.001	99.6%	44.0%	–	0.0806
adam lr=0.003	100.0%	44.0%	46	0.0070
adam lr=0.01	100.0%	42.5%	34	0.0013

Table 5: Stage 1 – optimiser and learning rate (60 epochs; “–” = never reached zero training error).

Setting	Train acc.	Val. acc.	Epochs to 0 train err.	Final train loss
relu	99.5%	45.6%	–	0.0233
tanh	99.9%	42.1%	46	0.0339
sigmoid	90.6%	41.1%	–	0.5108

Table 6: Stage 2 – activation function.

Setting	Train acc.	Val. acc.	Epochs to 0 train err.	Final train loss
cross-entropy	99.5%	45.6%	–	0.0233
MSE	85.3%	43.6%	–	0.1864

Table 7: Stage 3 – cost function.

Setting	Train acc.	Val. acc.	Epochs to 0 train err.	Final train loss
bs=16	56.5%	25.6%	–	8.8025
bs=32	100.0%	47.7%	38	0.0045
bs=64	99.5%	45.6%	–	0.0233
bs=128	99.6%	46.6%	–	0.0396
bs=256	98.5%	45.0%	–	0.1326

Table 8: Stage 4 – batch size.

Setting	Train acc.	Val. acc.	Epochs to 0 train err.	Final train loss
1x128 lr=0.5	100.0%	47.7%	38	0.0045
1x128 lr=0.1	99.5%	45.4%	–	0.0597
2x128 lr=0.5	57.0%	24.7%	–	1.9932
2x128 lr=0.1	100.0%	46.0%	57	0.0099
3x128 lr=0.5	1.6%	1.6%	–	4.1272
3x128 lr=0.1	100.0%	48.7%	45	0.0034
4x128 lr=0.5	33.5%	18.0%	–	2.9929
4x128 lr=0.1	100.0%	49.5%	34	0.0016
1x256	99.9%	50.1%	28	0.0057
1x512	100.0%	49.7%	27	0.0038

Table 9: Stage 5 – depth/width and learning rate.

Setting	Train acc.	Val. acc.	Epochs to 0 train err.	Final train loss
l2=0	99.9%	50.1%	28	0.0057
l2=0.0001	99.8%	49.7%	44	0.0130
l2=0.001	99.5%	53.6%	–	0.0842
l2=0.01	73.5%	48.1%	–	1.1224

Table 10: Stage 6 – L2 weight decay.

12 Discussion

Q1. Why does PLA underperform MLP?

- It draws one straight line per class, but these classes are not linearly separable.
- It cannot even fit its own training data (76.3% train accuracy).
- The step update has no loss or margin, so on this data it never settles.
- Result: 23.0% test accuracy vs 49.4% for the MLP.

Q2. Which hyperparameters mattered most?

- **Learning rate** by far. Validation accuracy ranged from 15.9%–45.6% across optimizer and lr settings.
- lr 0.5 broke batch size 16 (25.6%) and 3 hidden layers (1.6%).
- L2 weight decay helped by 3.5 points.
- Activation and loss: small differences, within noise (about ± 2 points).

Q3. Did SGD vs Adam affect convergence?

- Adam worked well over a wide lr range (10^{-3} to 10^{-2} , validation 42.5%–44.0%).
- SGD was very sensitive: lr 0.01 gave 32.3%, lr 0.5 gave 45.6%.
- Full batch GD was slowest (one update per epoch).
- Best SGD and best Adam differ by only 1.6 points, so neither is clearly better.

Q4. Did more hidden layers always help?

- No. At lr 0.5, 3 layers failed completely (1.6%).
- At lr 0.1, going from 1 to 4 layers gave about 4 points (45.4% to 49.5%), close to noise.
- One wide layer (50.1%) did as well, so I kept one hidden layer.
- Every stable model already reached about 100% train accuracy, so extra depth had nothing left to fix.

Q5. Did the MLP overfit? How to reduce it?

- Yes. Train accuracy 99.7%, test accuracy 49.4%.
- Without regularization, validation loss is lowest at epoch 8, then rises while training loss keeps falling.
- Used: L2 weight decay and early stopping.
- Could also try: data augmentation, dropout, more data, or a CNN.

Impact of tuning: untuned MLP 1.2%, tuned MLP 49.4%. During the search, validation accuracy went from 44.0% to 53.6%. The final model scores 56.0% on validation but 49.4% on test, because the settings were picked on the validation set. The test number is the honest one.

13 Conclusion

- The PLA reaches 23.0%. The tuned MLP reaches 49.4% (macro F1 48.8%).
- Learning rate was the most important hyperparameter. Deeper was not clearly better.
- Overfitting is the main problem left, because there are only about 38 training images per class.

14 Learning Outcomes

- Implemented the PLA and an MLP with backpropagation from scratch.
- Learned to tune hyperparameters on a separate validation set.
- Saw how learning rate, batch size and depth affect each other.
- Learned to read confusion matrices, ROC curves and convergence curves.

15 References

1. Stephen Marsland, “Machine Learning - An Algorithmic Perspective”, 2nd Edition, Chapman and Hall/CRC Machine Learning and Pattern Recognition Series, 2015.
2. Ethem Alpaydin, “Introduction to Machine Learning”, 3rd Edition, The MIT Press, 2014.
3. F. Rosenblatt, “The perceptron: a probabilistic model for information storage and organization in the brain”, Psychological Review, 65(6), 1958.
4. D. P. Kingma and J. Ba, “Adam: a method for stochastic optimization”, ICLR 2015.